

The Cambrian Explosion of Symmetric Techniques for Advanced Protocols

Léo Perrin

Inria, Paris
`leo.perrin@inria.fr`

2nd of September 2026



Cambrian Explosion?



In this talk

Is there a **revolution** going on in symmetric cryptography?

In this talk

Is there a **revolution** going on in symmetric cryptography?

absolutely not.

In this talk

Is there a **revolution** going on in symmetric cryptography?

absolutely not.

Design criteria are changing, **again.**

In this talk

Is there a **revolution** going on in symmetric cryptography?

Part 1.1-2

absolutely not.

Design criteria are changing, **again.**

Part 1.3

Part 1

- 1 What are symmetric primitives?
- 2 How do we build secure symmetric primitives?
- 3 On their design constraints

In this talk

Is there a **revolution** going on in symmetric cryptography?

Part 1.1-2

absolutely not.

Design criteria are changing, **again.**

Part 1.3

Part 2

Part 1

- 1 What are symmetric primitives?
- 2 How do we build secure symmetric primitives?
- 3 On their design constraints

Part 2

- 1 Securing computations vs. data
- 2 Arithmetization-Orientation?

In this talk

Is there a **revolution** going on in symmetric cryptography?

Part 1.1-2

absolutely not.

Conclusion

Design criteria are changing, **again.**

Part 1.3

Part 2

Part 1

- 1 What are symmetric primitives?
- 2 How do we build secure symmetric primitives?
- 3 On their design constraints

Part 2

- 1 Securing computations vs. data
- 2 Arithmetization-Orientation?

Outline

[0]

1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

3 A Revolution?

Plan of this Section

[0]

1 On Symmetric Primitives

2 "Advanced" Protocols: the Reason Behind Some Changes

3 A Revolution?

Plan of this Section

[0]

1 On Symmetric Primitives

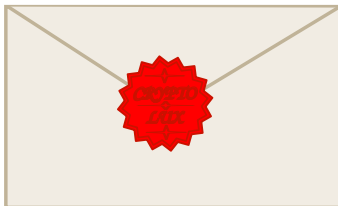
- What are symmetric primitives?
- How do we build secure symmetric primitives?
- On their Design Constraints

2 "Advanced" Protocols: the Reason Behind Some Changes

3 A Revolution?

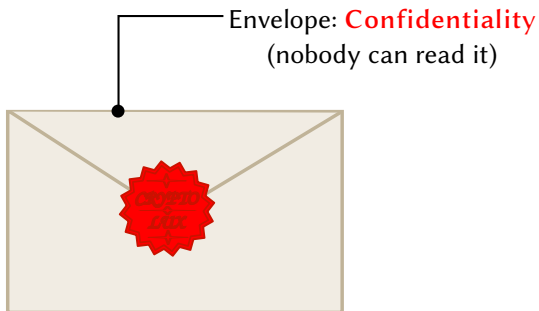
Securing Data

Usually, we secure **data** (at rest or in transit).



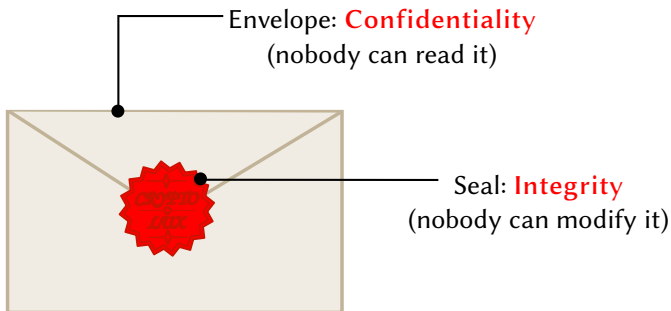
Securing Data

Usually, we secure **data** (at rest or in transit).



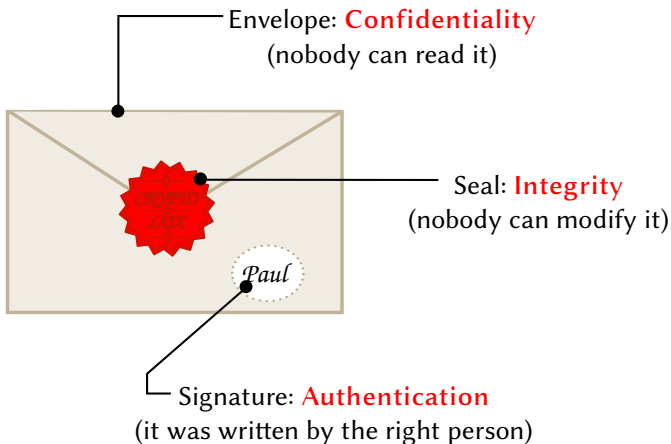
Securing Data

Usually, we secure **data** (at rest or in transit).



Securing Data

Usually, we secure **data** (at rest or in transit).



Definitions

Definition (Primitive)

A primitive is a very low level algorithm, a *basic brick* used to build larger things.

"Symmetric"

In **symmetric** cryptography, we consider that **key exchange** is a solved problem.

it's really not, see e.g. PQ crypto

Domain

We let $\mathbb{F}_q$ be a finite field. **Usually**, $q = 2^n$.

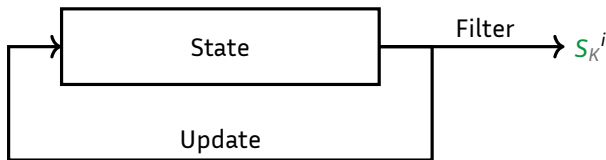
Stream Ciphers

Stream Cipher

Generates a keystream $S_K \in (\mathbb{F}_q)^*$ of arbitrary size which is then added to the plaintext (block by block) to obtain the ciphertext ($\approx$ one time pad). S_K should be **indistinguishable** from a random sequence.

In practice:

- 1 a state of $\mathbb{F}_q^n$ is initialized using the secret key K
- 2 at each "clock cycle", the state is **updated** and a keystream block is **extracted**.



Design

Pick n , design good filter/update combination

Examples: **Chacha20**, **Snow-3G**...

Hash Functions

Hash Function

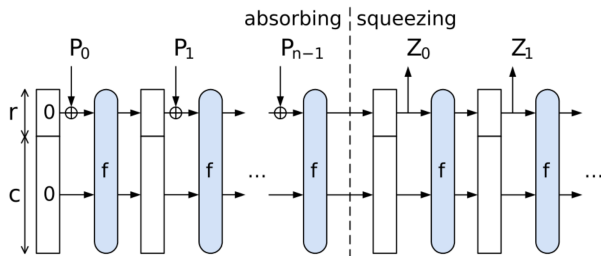
A function $H : (\mathbb{F}_q)^* \rightarrow \mathbb{F}_q^n$ mapping arbitrarily sized inputs to fixed-size outputs.

Preimage resistance given y , can't find x such that $H(x) = y$

2nd preimage resistance given x , can't find x' such that $H(x) = H(x')$

Collision resistance find (x, x') such that $H(x) = H(x')$.

In practice, these days, uses the **sponge construction**



Design

Pick, r , c , and design a good permutation f .

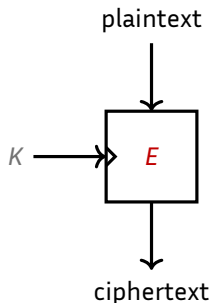
Examples: **SHA-3**, **Blake...**

Block Ciphers

Block Cipher

Family $E_K : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ of bijections operating on blocks of fixed size.

Basically useless without a **mode of operation**.



Design

We will see!

Examples: **AES**, **Skinny**, **PRESENT**...

Why bother?

Why would anyone need a “symmetric” primitive?

AES, SHA-3, Chacha20, Skinny, Snow-3G, Poly-1305, PRESENT, Blake, GHASH...

Efficiency

Security

Plan of this Section

[0]

1 On Symmetric Primitives

- What are symmetric primitives?
- How do we build secure symmetric primitives?
- On their Design Constraints

2 "Advanced" Protocols: the Reason Behind Some Changes

3 A Revolution?

Usual Approach

The Big Trade Secret of Symmetric Cryptographers©

If you squint hard enough, **everything is a block cipher**

Hash function To build a permutation, take a block cipher and set $K = 0$

Stream cipher Use $+1$ as your update, and a full block cipher call as the filter (counter mode).
OR.

Put a block cipher in the **update** function

What is "Secure"?

Dear audience, what is a **secure** block cipher?

What is **secure** varies

- Has the primitive been **analyzed**?
 - By the designers? **Design quality**
 - By third parties? **Analyzability**
- Should the primitive work in many context? **modularity vs. single use**

How do we define the **security** that the primitive must provide?

What are the relevant forms of cryptanalysis?

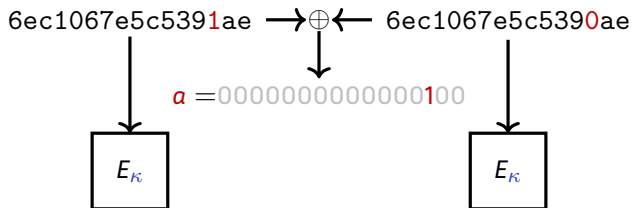
On the Importance of Cryptanalysis

Let us look at a typical cryptanalysis technique:
the **differential attack**.

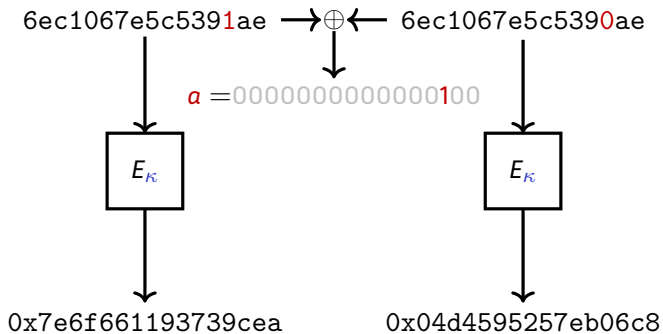
Differential Attacks

$$\begin{array}{ccc} 6ec1067e5c5391ae & \xrightarrow{\oplus} & 6ec1067e5c5390ae \\ & \downarrow & \\ a = 000000000000000100 & & \end{array}$$

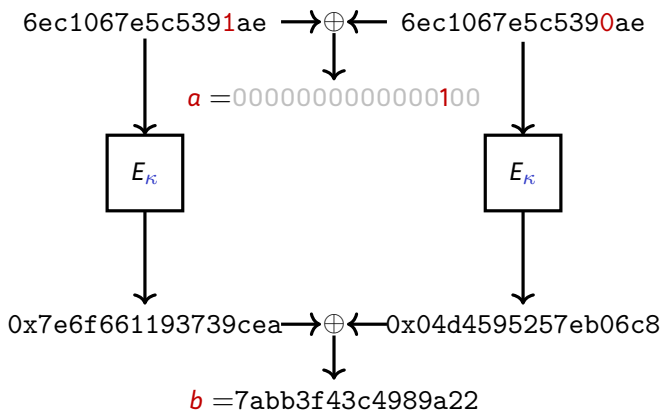
Differential Attacks



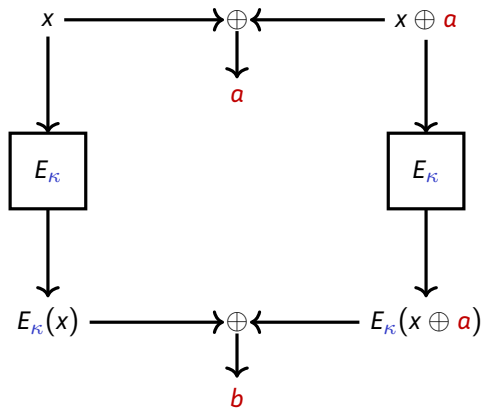
Differential Attacks



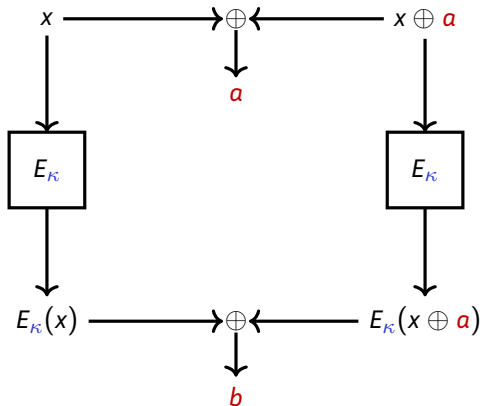
Differential Attacks



Differential Attacks



Differential Attacks

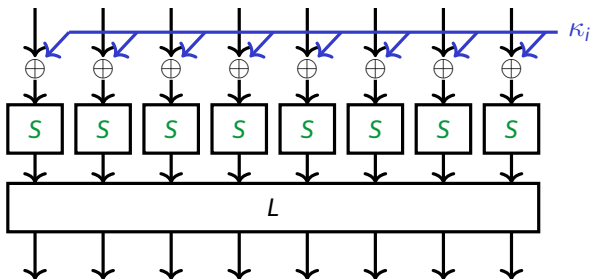


Differential Attack

If there are many x such that $E_{\kappa}(x) \oplus E_{\kappa}(x \oplus a) = b$, then the cipher is **not secure**.

Basic Block Cipher Structure

How do we build block ciphers that prevent such attacks (as well as others)?



Substitution-Permutation Network

Such a block cipher iterates the round function above several times. **S** is the **Substitution Box** (S-Box).

Choosing the Components

Importance of the S-Box

If S is such that

$$S(x) \oplus S(x \oplus a) = b$$

does not have many solutions x for all (a, b) then the cipher may be proved secure against differential attacks. Optimal S-boxes are **Almost Perfect Nonlinear (APN)**.

Importance of the linear layer L

We want the following quantity (**branching number**) to be as high as possible

$$\min (\text{wt}(x) + \text{wt}(L(x)), x \neq 0) .$$

Optimal linear layers correspond to **Maximum Distance Separable (MDS)** codes.

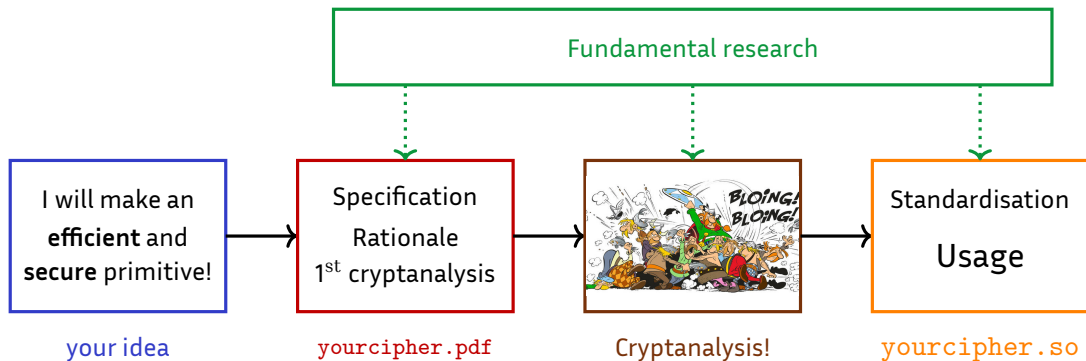
In **academic** papers presenting new block ciphers,
the choice of S and L is carefully explained.

To Build a Cipher

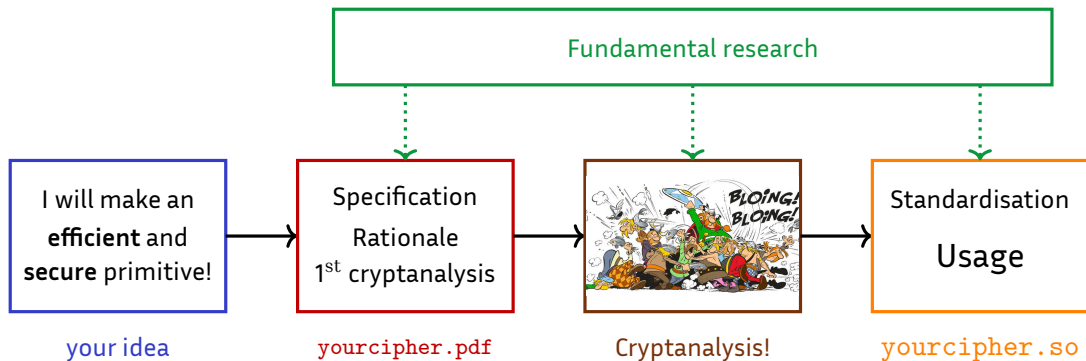
- 1 Regardless of which primitive you build, you actually need a secure block cipher.
- 2 To build a secure block cipher, you need a round function, meaning:
 - A highly “non-linear” S-box
 - A linear layer with a “high diffusion”
- 3 Using **dedicated cryptanalysis** and **well known techniques**, estimate the number of rounds needed for security
- 4 Choose the final number of rounds, taking into account the **security margin** ($\times 1.2?$ $+2?$)

Voilà!

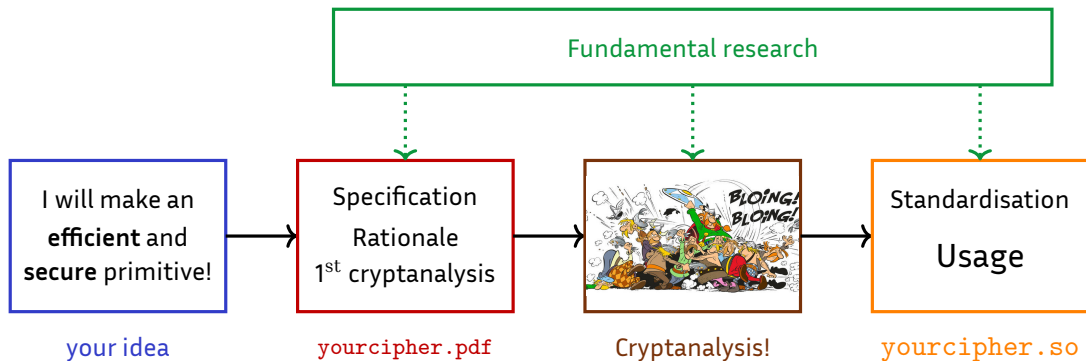
The Cryptography Research Pipeline



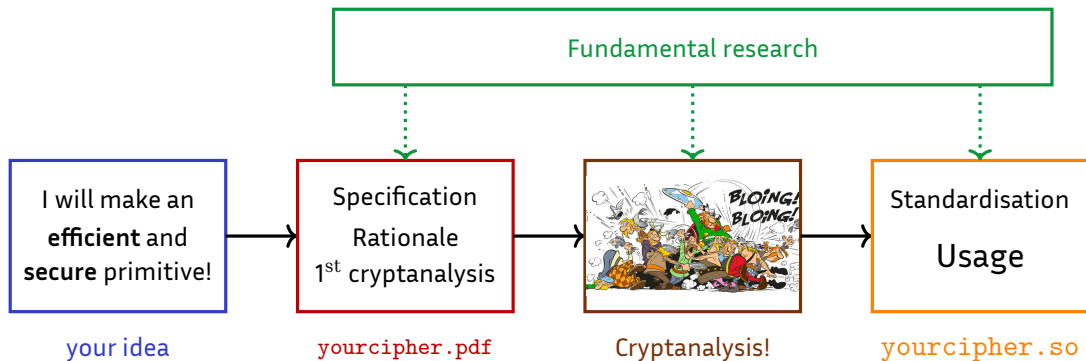
The Cryptography Research Pipeline



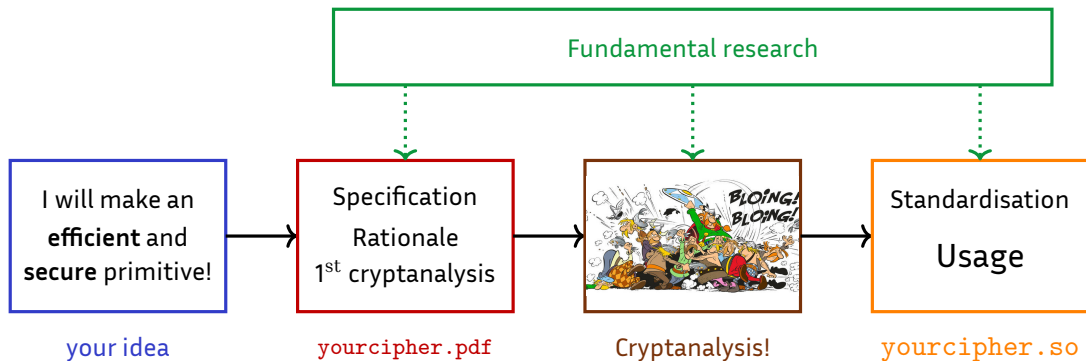
The Cryptography Research Pipeline



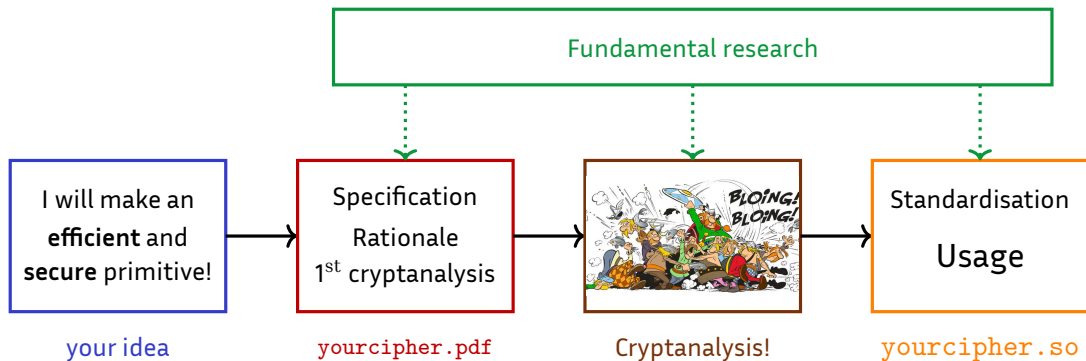
The Cryptography Research Pipeline



The Cryptography Research Pipeline



The Cryptography Research Pipeline



But is your cipher **efficient** enough to make it to the final stage?

What is "Efficient"?

Dear audience, what is an **efficient** block cipher?

What is **efficient** varies

- What are the operations that we **can** use?
basic logical gates? CPU instructions? AES round?...
- What are the associated **costs**?
throughput? physical area? ram consumption? number of masked multiplication?...

What is the intended **execution context**?

Plan of this Section

[0]

1 On Symmetric Primitives

- What are symmetric primitives?
- How do we build secure symmetric primitives?
- On their Design Constraints

2 "Advanced" Protocols: the Reason Behind Some Changes

3 A Revolution?

Web Encryption

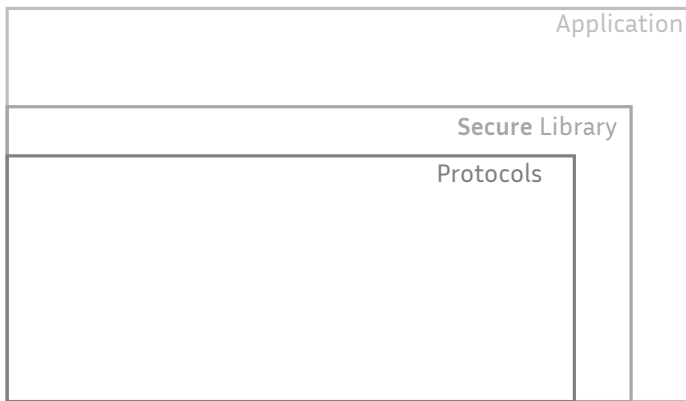
Application



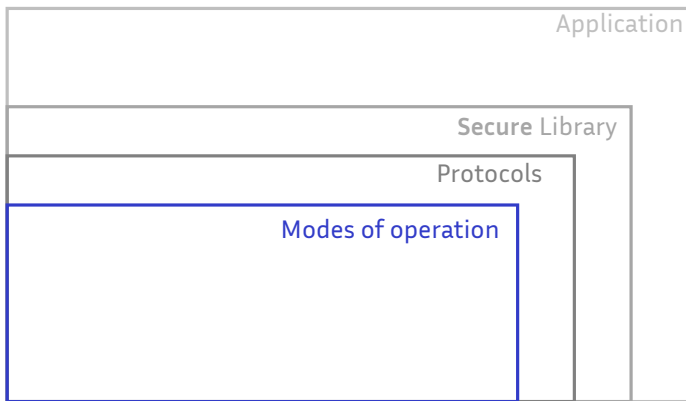
Web Encryption



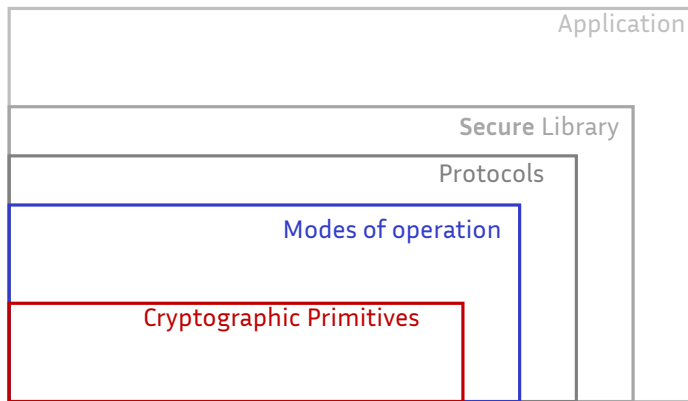
Web Encryption



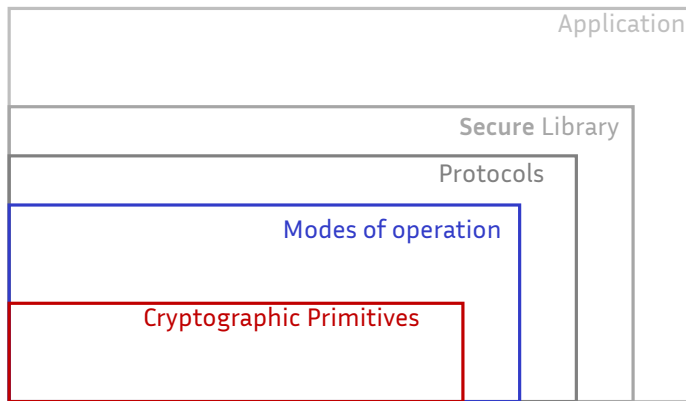
Web Encryption



Web Encryption

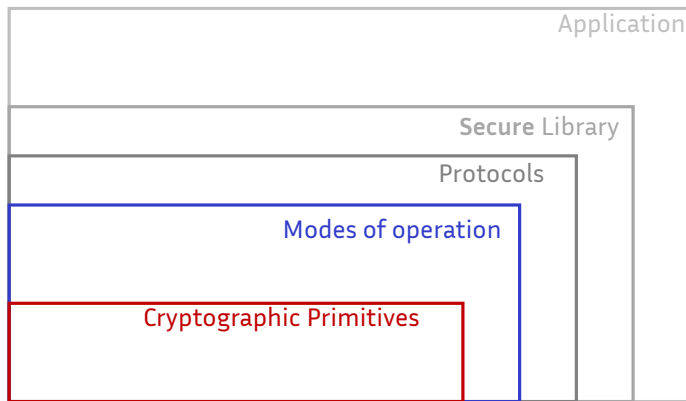


Web Encryption



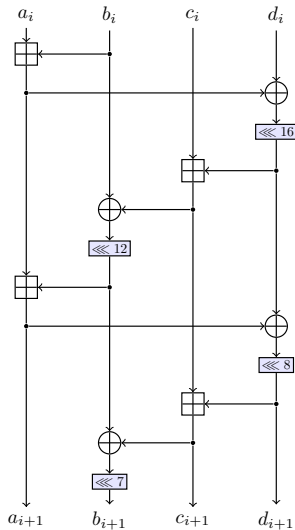
- We want **software efficient** (computer and smartphone but not micro-controllers) efficient **AEAD** for packets of a few tens to a few billion bytes.

Web Encryption



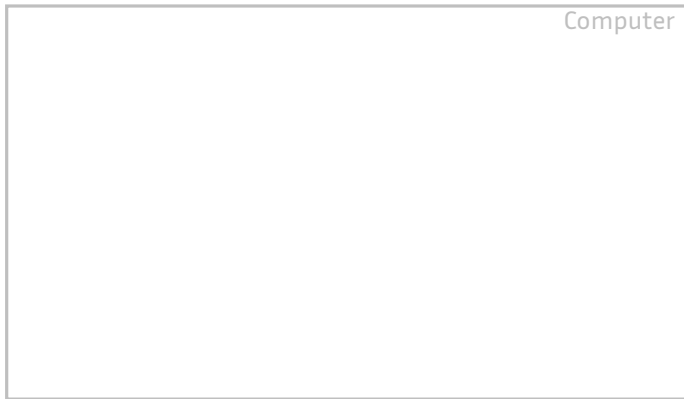
- We want **software efficient** (computer and smartphone but not micro-controllers) efficient **AEAD** for packets of a few tens to a few billion bytes.
- AES-GCM; Chacha-poly1305.

What Chacha looks like



- Addition / Rotation / XOR
- 256-bit key
- 512-bit state
- Defined over 32-bit words

RAM Encryption



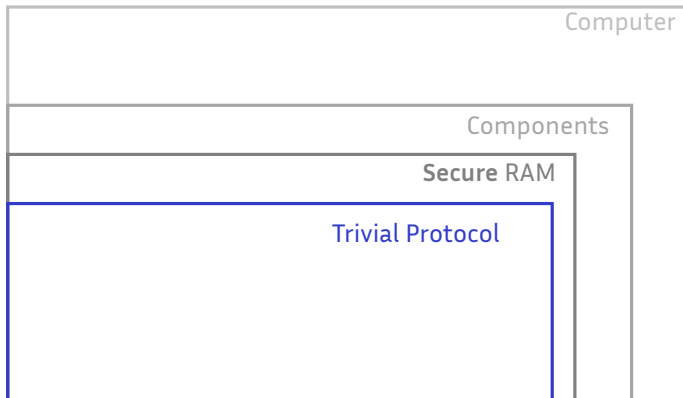
RAM Encryption



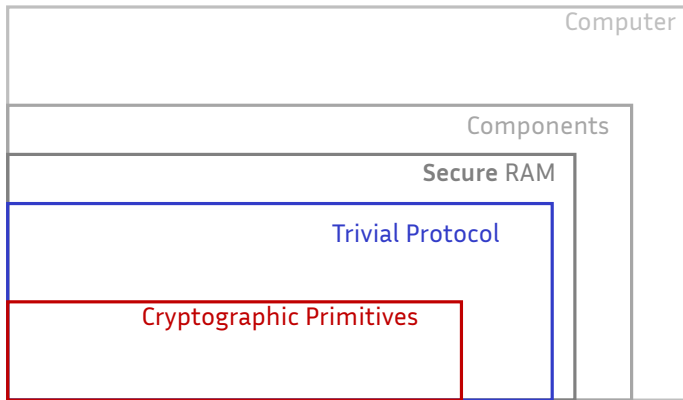
RAM Encryption



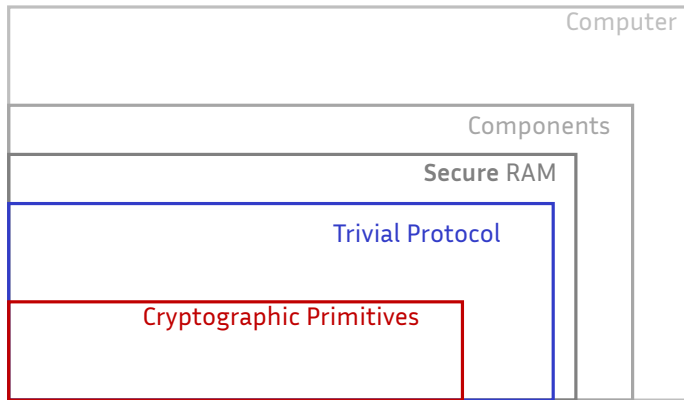
RAM Encryption



RAM Encryption

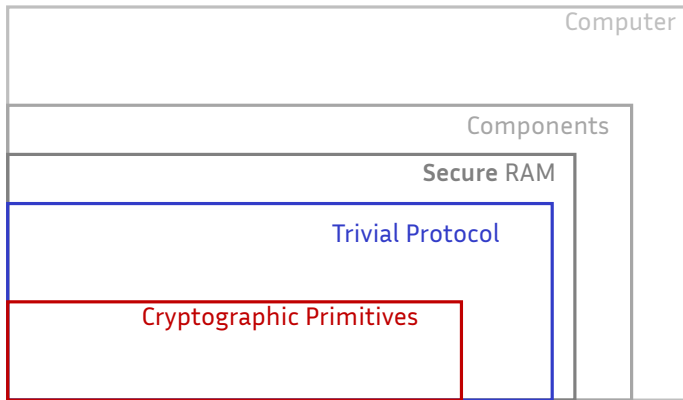


RAM Encryption



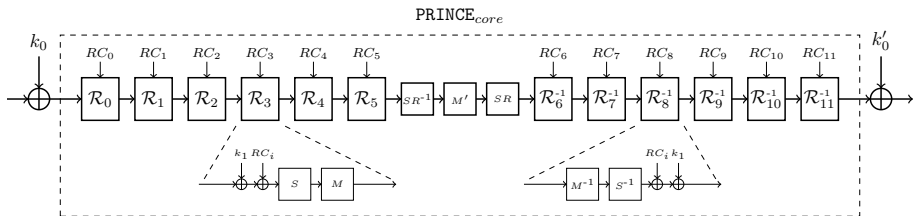
- We want **very low latency block encryption** for specific (and small) block sizes.

RAM Encryption



- We want **very low latency block encryption** for specific (and small) block sizes.
- PRINCE? QARMA? not so clear at this stage.

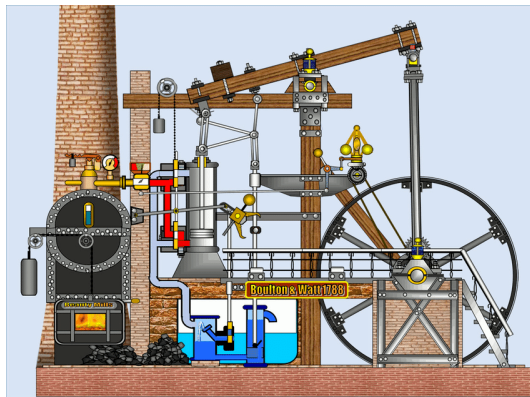
What PRINCE looks like



- 64-bit block size; 128-bit key size ($\approx$)
- 4-bit S-box optimized for hardware
- 2 different 16×16 matrices of $\mathbb{F}_2$, also optimized for hardware
- FX construction
- "α-reflexion": inverse rounds used in the second half

Some Constants

There are many **different** "big machines", and

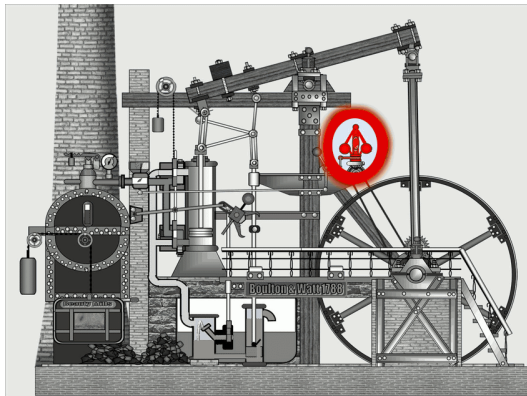


source: https://www.researchgate.net/figure/The-Steam-Engine-of-James-Watt-and-Mathew-Bolton-15_fig2_347657192

Some Constants

There are many **different** "big machines", and

a symmetric primitive is a very **small** (but crucial) cog in a very big machine,



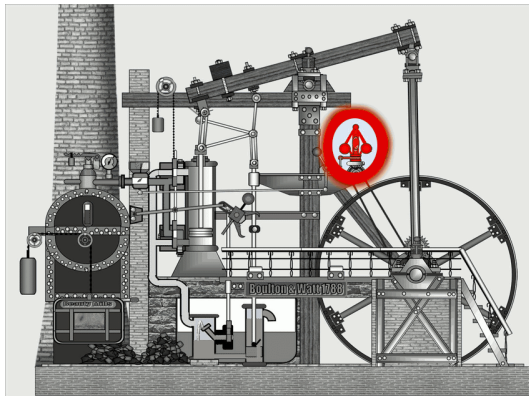
source: https://www.researchgate.net/figure/The-Steam-Engine-of-James-Watt-and-Mathew-Bolton-15_fig2_347657192

Some Constants

There are many **different** "big machines", and

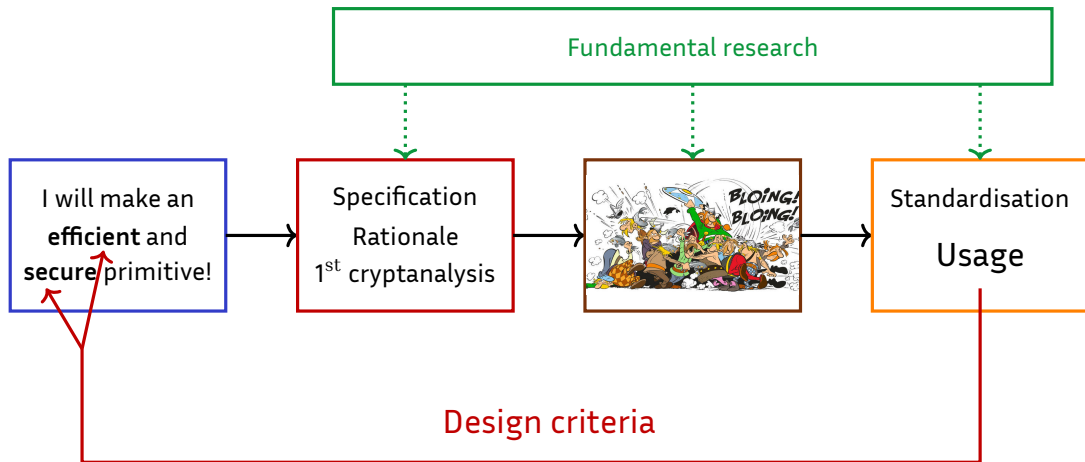
a symmetric primitive is a very **small** (but crucial) cog in a very big machine,

this has a **huge influence** on what the primitive looks like.



source: https://www.researchgate.net/figure/The-Steam-Engine-of-James-Watt-and-Mathew-Bolton-15_fig2_347657192

To Build a Cipher (full)



Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)



CRYPTANALYSIS TIME

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)

August 2019 start of second round (32 candidates left)

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)

August 2019 start of second round (32 candidates left)



CRYPTANALYSIS TIME

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)

August 2019 start of second round (32 candidates left)

March 2021 start of third (final) round (10 candidates left)

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)

August 2019 start of second round (32 candidates left)

March 2021 start of third (final) round (10 candidates left)



not much cryptanalysis...

Example: NIST's Lightweight Cryptography (not) Competition

March 2014 First mentions of a LW standardization process

2015-2018 Calls for comments (several iterations)

August 2018 Call for submissions

March 2019 Submission deadline/start of first round (62 candidates)

August 2019 start of second round (32 candidates left)

March 2021 start of third (final) round (10 candidates left)

February 2023 NIST chose **ASCON**

Take-Away

What is the point of having symmetric primitive?

High performance Designing a protocol is **pointless** if the symmetric primitive is **not efficient** enough, and compromises are not really an option.

High Security Designing a protocol is **pointless** if the symmetric primitive is **not secure** enough, and compromises are not really an option*.

* despite what a busy decision maker might think
ask the dutch railway company :D

Plan of this Section

[0]

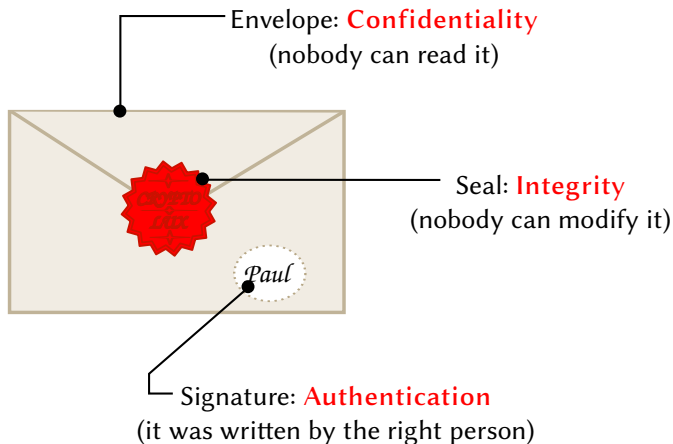
1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

3 A Revolution?

Securing Data

Usually, we secure **data** (at rest or in transit).



Securing Computation

More and more protocols intend to secure **computations**.

FHE Fully **H**omomorphic **E**ncryption

MPC Multi **P**arty **C**omputations

ZK-* Zero **K**nowledge- [proof, argument...]

Plan of this Section

[0]

1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- Zero-Knowledge
- One Approach to Rule Them All (?): Arithmetization

3 A Revolution?

FHE

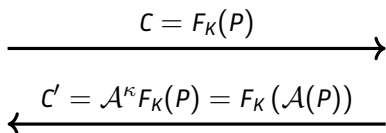
Goal

Allow a third party to perform some operations on encrypted ciphertext that correspond to meaningful operations on the corresponding plaintext.

A form of commutation

Alice

Bob



F_K is a homomorphic cipher,
not a block cipher!

An example of (not F)HE

XOR-ing a constant to a ciphertext obtained using a stream cipher XORs the same constant in the plaintext:

$$C \oplus t = (P \oplus K) \oplus t = (P \oplus t) \oplus K$$

The Symmetric Crypto They Need: Transciphering

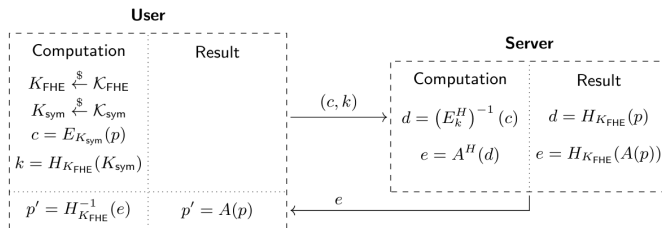


Fig. 1: The principle of transciphering, where E is a symmetric cipher (with secret key K_{sym} sampled from the space $\mathcal{K}_{\text{sym}}$), H is a fully homomorphic cipher (with private key K_{FHE} sampled from the space $\mathcal{K}_{\text{FHE}}$), E^H is a homomorphic evaluation of E , A corresponds to some arbitrary operations, and A^H to their homomorphic evaluation.

A symmetric encryption
 algorithm with a
 high throughput when
 evaluated
 homomorphically

source: *Transistor: a TFHE-friendly Stream Cipher*

<https://eprint.iacr.org/2025/282>

The case of TFHE

Operates on $\mathbb{Z}/m\mathbb{Z}$, where m can be anything, though: more efficient if m is smaller.

Operations allowed

Linear Combinations $\sum_i \alpha_i x_i$, where the α_i are constant while x_i is input/key dependent.

- Costs almost nothing in terms of time/communication complexity...
- But **noise** increases

PBS (Programmable BootStrap) $y \leftarrow S(x)$

- Very time consuming...
- .. But resets the noise to a **base level**
- Can be composed with **arbitrary table lookups!**

(not T)FHE operates differently, but **noise** is still present

Examples of stream ciphers for transciphering

Elisabeth-4 [CHMS22]

$q = 2^4$ Can be linearized [GBJR23]

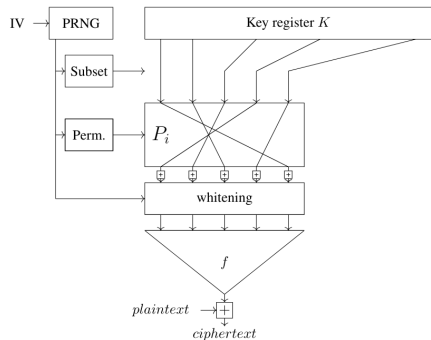


Fig. 1: The group filter permutator design

· ASTA

$q = 2$ or large prime

Many, many variants (Rasta, Dasta, Pasta, Masta)

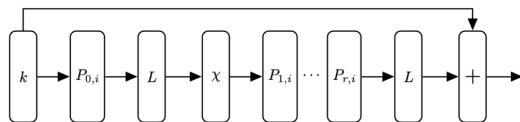


Figure 2: Generation of i -th block of DASTA.

source: Dasta – Alternative Linear Layer for Rasta

Plan of this Section

[0]

1 On Symmetric Primitives

2 "Advanced" Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- Zero-Knowledge
- One Approach to Rule Them All (?): Arithmetization

3 A Revolution?

Multi-Party Computations

Goal

Allow multiple parties to evaluate a function together even if some parties are not trustworthy.

Example

Shamir's secret sharing: $n + 1$ points are necessary to interpolate a degree n function.

Applications

- Masking (the side-channel attack counter-measure)
- MPC-in-the-head paradigm (e.g. for Picnic signatures)
- Trojan resilience
- ...

The SymCry They Need: a Lot of Different Things

Masking-friendly (Tweakable) BCs. Low number of multiplications in the underlying field.

Examples: `small-pSquare` [GMM⁺24b], `Fantomas` [GLSV15],...

Picnic-friendly BCs. Low number of multiplications, security only for a single query.

Example: `LowMC` [GRR⁺16]

Trojan Resilient BCs Reliance only on linear operations (but over different ring/fields).

Example: `MOE` [BFL⁺21]

Pseudo-Correlated Functions

A new challenger!

At the start of some MPC protocols, it is necessary to share some bits that are correlated between the participants.

Very low multiplicative depth · Very low data complexity · **Only known plaintext, no chosen plaintext!**

Crypto DarkMatter [BIP⁺18]

$$F_k(x) := \left(\sum_{i=0}^{n-1} k_i x_i \bmod 2 + \sum_{i=0}^{n-1} k_i x_i \bmod 3 \right) \bmod 2, \quad \text{for } x \in \{0, 1\}^n.$$

VDLPN [BCG⁺20]

$$f_k(x) = \bigoplus_{i=1}^D \bigoplus_{j=1}^w \bigwedge_{\ell=1}^i (x_{i,j,\ell} \oplus k_{i,j,\ell}),$$

Plan of this Section

[0]

1 On Symmetric Primitives

2 "Advanced" Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- Zero-Knowledge
- One Approach to Rule Them All (?): Arithmetization

3 A Revolution?

Zero-Knowledge

Principle

I want to convince you I know a secret **without revealing it**

A generic goal

To be able to prove/argue that a function was evaluated correctly without revealing its input.
Ex: Sudoku time!

Applications

- Offload computation to untrusted 3rd parties
- Electronic vote (mixnets)
- Signatures (e.g. with Solid)
- BLOCKCHAIN!!!

The Symetric Crypto They Need: Easily Verifiable Hash Functions

(what follows is a simplification)

They want **hash functions** where the round function has a...

Low Degree Verification

A function $F : \mathbb{F}_q \rightarrow \mathbb{F}_q$ admits a *low degree verification* if there exists a low degree function $G : \mathbb{F}_q^2 \rightarrow \mathbb{F}_q$ such that

$$F(x) - z = 0 \Leftrightarrow G(x, z) = 0.$$

Direct evaluation if F has a low degree, we are fine: $G(x, z) = F(x) - z$

Indirect evaluation if F^{-1} has a low degree, then $G(x, z) = x - F^{-1}(z)$

Generalization it is sufficient that F is **CCZ-equivalent** to a low degree function.

Example 1/4: MiMC

MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity

Martin Albrecht¹, Lorenzo Grassi³, Christian Rechberger^{2,3}, Arnab Roy², and Tyge Tiessen²

¹ Royal Holloway, University of London, UK
martinalbrecht@googlemail.com

² DTU Compute, Technical University of Denmark, Denmark
{arroy, crec, tyti}@dtu.dk

³ IAIK, Graz University of Technology, Austria
{christian.rechberger, lorenzo.grassi}@iaik.tugraz.at

Published at ASIACRYPT'16;

<https://eprint.iacr.org/2016/492.pdf>

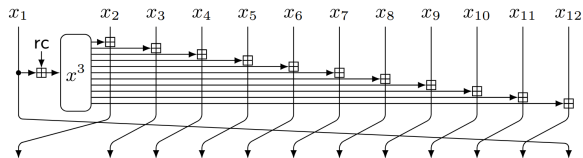
- Base field: $\mathbb{F}_q$, where e.g. $q = 2^{257}$
- Round function:

$$R_i \begin{cases} \mathbb{F}_q & \rightarrow \mathbb{F}_q \\ x & \mapsto (x + c_i)^3 \end{cases}$$

where the *round constants* c_i have been generated randomly.

- Number of rounds: $\ell \approx 90$

Example 2/4: gMiMC

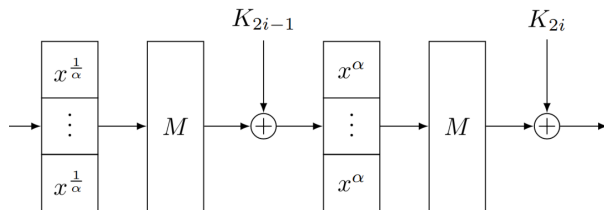


Published at ESORICS'19;
Albrecht, Perrin, Ramacher, Rechberger, Rotaru, Roy, Schofnegger

<https://eprint.iacr.org/2019/397.pdf>

- Base field: $\mathbb{F}_q$, where $q = 2^n$ or $q = p \geq 2^n, n \geq 64$
- Round function: see left
- Number of rounds: $\ell > 170$

Example 3/4: Rescue



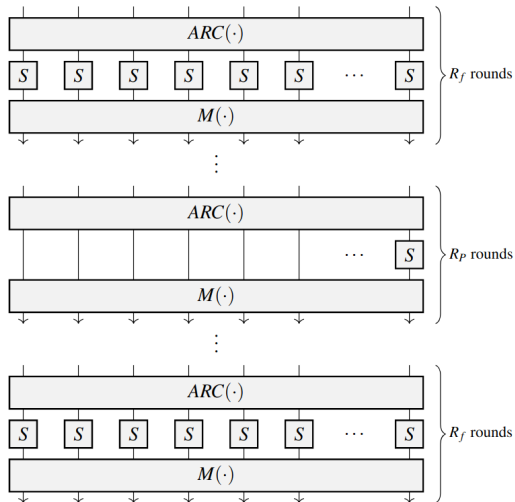
Published at ToSC'20(3);
Aly, Ashur, Ben-Sasson, Dhooghe, Szepieniec

<https://tosc.iacr.org/index.php/ToSC/article/view/8695/8287>

- Base field: $\mathbb{F}_q$, where
 $q = p \geq 2^n, n \geq 64$
- Round function: see left; $\alpha = 3$
and M is a linear permutation of
 $\mathbb{F}_q^t$.
- Number of rounds: $\ell \approx 10$

Verification: $P_i(x_i) == Q_i(x_{i+1})$, where P_i is a half round,
and Q_i is the **inverse** of the other half!

Example 4/4: Poseidon



Published at USENIX'21;
 Grassi, Khovratovich, Rechberger, Roy, Schofnegger
<https://eprint.iacr.org/2019/458.pdf>

- Base field: $\mathbb{F}_q$, where $q = p \geq 2^n$, $n \geq 64$
- Round function: $S(x) = x^3$, ARC add a round constant, and M is a linear permutation of $\mathbb{F}_q^t$.
- Number of rounds: $\ell = R_f + R_p \approx 50$

Cambrian explosion!



Plan of this Section

[0]

1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

- (Fully) Homomorphic Encryption
- Multi-Party Computations
- Zero-Knowledge
- One Approach to Rule Them All (?): Arithmetization

3 A Revolution?

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

1

2

3 $t_2 = t_1 \times t_1$

4 $t_3 = t_2 \times t_2$

5 $t_4 = t_3 \times t_3$

6 $t_5 = t_2 \times t_4$

7

8

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

This verification costs 4 R1CS constraints

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

This verification costs 4 R1CS constraints

- How to turn a computation into an arithmetic circuit depends on the operations allowed
- Its cost is also arithmetization-dependent—though low degree is usually welcome!

A Basic Example of Arithmetization

Verifying if $y = c(ax + b)^{10} + x$ in R1CS

$$1 \quad t_0 = ax$$

$$2 \quad t_1 = t_0 + b$$

$$3 \quad t_2 = t_1 \times t_1$$

$$4 \quad t_3 = t_2 \times t_2$$

$$5 \quad t_4 = t_3 \times t_3$$

$$6 \quad t_5 = t_2 \times t_4$$

$$7 \quad t_6 = ct_5$$

$$8 \quad y = t_6 + x$$

This verification costs 4 R1CS constraints

- How to turn a computation into an arithmetic circuit depends on the operations allowed
- Its cost is also arithmetization-dependent—though low degree is usually welcome!
- Arithmetization over a field of odd size → nonbinary ciphers

A not basic at all example of arithmetization

The cost of each operation depends on the arithmetization!
 Plonk $\neq$ R1CS

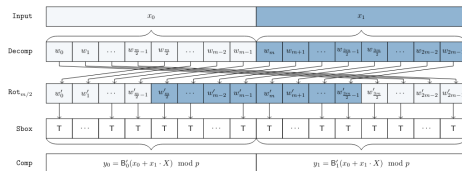
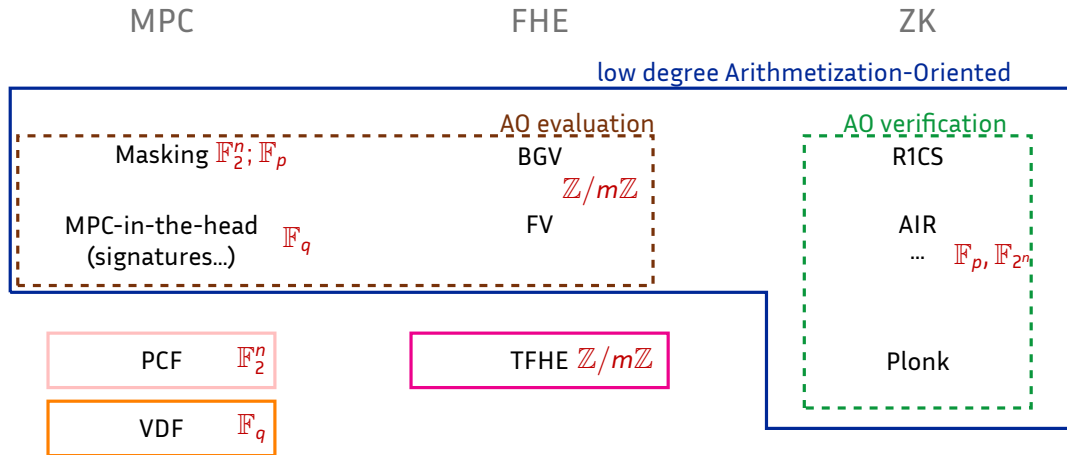


Figure 3: The Bar layer $B' : \mathbb{F}_{p^n} \rightarrow \mathbb{F}_{p^n}$ for $n = 2$ in detail, including the decomposition, the rotation, the S-box, and the composition.

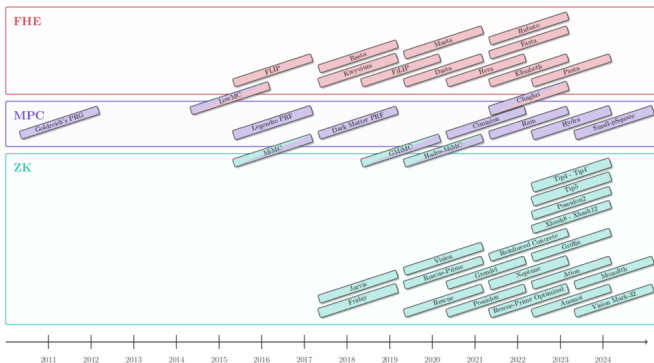
source: *Skyscraper: Fast Hashing on Big Primes*,
<https://eprint.iacr.org/2025/058.pdf>

Symmetric Techniques for Advanced Protocols



A Cambrian Explosion

credit: Clémence Bouvier [Bou23]



<https://stap-zoo.com/>

An increased diversity of
design criteria leads to a
Cambrian explosion of new
symmetric primitives!

Plan of this Section

[0]

1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

3 A Revolution?

Plan of this Section

[0]

1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

3 A Revolution?

- What Is and Isn't Specific to STAPs

- Conclusion

Design Process

Once upon a time, a symmetric cryptographer designed a new symmetric primitive

- Long initial phase to figure out what operations are the most **efficient**
- **New tricks** for the initial security analysis
- Tight **collaboration with implementers** $\neq$ the people doing the cryptanalysis
- Multiple **iterations** during the design to maximize efficiency

Dear audience, which specific primitive am I talking about?
all of them!

Opinion 1

STAPs are nothing special: we (symmetric people) need to do what we always did.

Underlying Alphabet

$\mathbb{F}_q$ and $\mathbb{F}_2^n$ are not the same!

- Indeed. In particular, q prime means no Frobenius automorphisms, meaning linear operations are only **constant multiplications**.
- Protocol flexibility means STAPs are often in fact **primitives generators**, i.e., algorithms **generating** primitives.
- Low degree arithmetization implies low degree algebraic modeling

beware of algebraic attacks!

(there already was a mass extinction event, the **Freelunch attack**
(I'm pretty sure **Morten** will tell you more!)

Opinion 2

Working over $\mathbb{F}_q$ (especially if low degree arithmetizations are needed) and the need for primitive generators introduce **new cryptanalysis vectors**, **but design approaches will rely on tried and true methods**.

Primitive Overdose



credit: Diego Delso, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=108259695>

Cryptanalysis has not followed the design
explosion

Potential Explanations

- Not all primitives are designed to **ease analysis**
- So many (too many) minor variants...

Opinion 3

- 1 We need more **cryptanalysis!**
- 2 We must become better at handling **primitive generators.**

Plan of this Section

[0]

1 On Symmetric Primitives

2 “Advanced” Protocols: the Reason Behind Some Changes

3 A Revolution?

- What Is and Isn't Specific to STAPs

- Conclusion

What Have We Learnt?

- The shape of symmetric primitives has always been dictated by **external constraints**
- New **advanced protocols** intended to secure **computations** need symmetric primitives
- The **constraints** they impose on the designs imply an **unusual look at first glance**
- The **constraints** they impose **vary**, and **we should be careful about lumping them together**
- A lot of new sets of constraints $\implies$ a lot of new designs (**Cambrian explosion**)...
- ... but not enough cryptanalysis.

We need more cryptanalysis!

Design criteria dictate the work of symmetric cryptographers
they always have, and always will!

Thank you!

-  Abdelrahman Aly, Tomer Ashur, Eli Ben-Sasson, Siemen Dhooghe, and Alan Szepieniec.
Design of symmetric-key primitives for advanced cryptographic protocols.
IACR Trans. Symm. Cryptol., 2020(3):1–45, 2020.
-  Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner.
Ciphers for MPC and FHE.
In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 430–454, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
-  Jules Baudrin, Sonia Belaïd, Nicolas Bon, Christina Boura, Anne Canteaut, Gaëtan Leurent, Pascal Paillier, Léo Perrin, Matthieu Rivain, Yann Rotella, and Samuel Tap.
Transistor: a TFHE-friendly stream cipher.
Cryptology ePrint Archive, Paper 2025/282, 2025.
-  Clémence Bouvier, Pierre Briaud, Pyrros Chaidos, Léo Perrin, Robin Salen, Vesselin Velichkov, and Danny Willems.

New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode.

In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 507–539, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.



Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl.
Correlated pseudorandom functions from variable-density lpn.

In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1069–1080. IEEE, 2020.



Olivier Bronchain, Sebastian Faust, Virginie Lallemand, Gregor Leander, Léo Perrin, and François-Xavier Standaert.

MOE: Multiplication operated encryption with trojan resilience.

IACR Trans. Symm. Cryptol., 2021(1):78–129, 2021.



Dan Boneh, Yuval Ishai, Alain Passelègue, Amit Sahai, and David J. Wu.

Exploring crypto dark matter: New simple PRF candidates and their applications.

In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part II*, volume 11240 of *LNCS*, pages 699–729, Panaji, India, November 11–14, 2018. Springer, Cham, Switzerland.

-  Clémence Bouvier.
Cryptanalysis and design of symmetric primitives defined over large finite fields.
Theses, Sorbonne Université, November 2023.
-  Anne Canteaut, Sergiu Carpov, Caroline Fontaine, Tancrede Lepoint, María Naya-Plasencia, Pascal Paillier, and Renaud Sirdey.
Stream ciphers: A practical solution for efficient homomorphic-ciphertext compression.
In Thomas Peyrin, editor, *FSE 2016*, volume 9783 of *LNCS*, pages 313–333, Bochum, Germany, March 20–23, 2016. Springer Berlin Heidelberg, Germany.
-  Mingyu Cho, Woohyuk Chung, Jincheol Ha, Jooyoung Lee, Eun-Gyeol Oh, and Mincheol Son.
Frast: Tffe-friendly cipher based on random s-boxes.
IACR Transactions on Symmetric Cryptology, 2024(3):1–43, Sep. 2024.
-  Jihoon Cho, Jincheol Ha, Seongkwang Kim, ByeongHak Lee, Joohee Lee, Jooyoung Lee, Dukjae Moon, and Hyojin Yoon.
Transciphering framework for approximate homomorphic encryption.
In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 640–669, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.

-  **Orel Cosseron, Clément Hoffmann, Pierrick Méaux, and François-Xavier Standaert.**
Towards case-optimized hybrid homomorphic encryption - featuring the elisabeth stream cipher.
In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part III*, volume 13793 of *LNCS*, pages 32–67, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.
-  **Christoph Dobraunig, Lorenzo Grassi, Anna Guinet, and Daniël Kuijsters.**
Ciminion: Symmetric encryption based on Toffoli-gates over large finite fields.
In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 3–34, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
-  **Henri Gilbert, Rachelle Heim Boissier, Jérémy Jean, and Jean-René Reinhard.**
Cryptanalysis of elisabeth-4.
In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part III*, volume 14440 of *LNCS*, pages 256–284, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
-  **Lorenzo Grassi, Yonglin Hao, Christian Rechberger, Markus Schofnegger, Roman Walch, and Qingju Wang.**

Horst meets fluid-SPN: Griffin for zero-knowledge applications.

In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 573–606, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.



Lorenzo Grassi, Reinhard Lüftenegger, Christian Rechberger, Dragos Rotaru, and Markus Schofnegger.

On a generalization of substitution-permutation networks: The HADES design strategy.

In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 674–704, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.



Vincent Grosso, Gaëtan Leurent, François-Xavier Standaert, and Kerem Varici.

LS-designs: Bitslice encryption for efficient masked software implementations.

In Carlos Cid and Christian Rechberger, editors, *FSE 2014*, volume 8540 of *LNCS*, pages 18–37, London, UK, March 3–5, 2015. Springer Berlin Heidelberg, Germany.



Lorenzo Grassi, Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert.

Generalized feistel ciphers for efficient prime field masking - full version.

Cryptology ePrint Archive, Report 2024/431, 2024.

 Lorenzo Grassi, Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert.

Generalized feistel ciphers for efficient prime field masking - full version.

Cryptology ePrint Archive, Paper 2024/431, 2024.

 Lorenzo Grassi, Christian Rechberger, Dragos Rotaru, Peter Scholl, and Nigel P. Smart.

MPC-friendly symmetric key primitives.

In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 430–443, Vienna, Austria, October 24–28, 2016. ACM Press.

 Clément Hoffmann, Pierrick Méaux, and François-Xavier Standaert.

The patching landscape of elisabeth-4 and the mixed filter permutator paradigm.

In Anupam Chattopadhyay, Shivam Bhasin, Stjepan Picek, and Chester Rebeiro, editors, *INDOCRYPT 2023, Part I*, volume 14459 of *LNCS*, pages 134–156, Goa, India, December 10–13, 2023. Springer, Cham, Switzerland.

Plan of this Section

[0]

4 Examples of Primitives

Plan of this Section

[0]

4 Examples of Primitives

■ FHE

■ MPC

■ ZK

TFHE: corresponding stream ciphers

Elisabeth-4 [CHMS22] ; $q = 2^4$

Uses a constant key register on which index-dependent non-linear functions are applied.

Can be linearized [GBJR23]

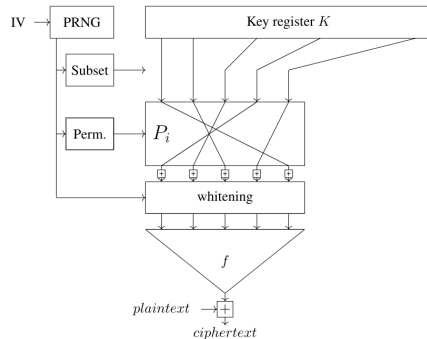


Fig. 1: The group filter permutator design

source: *Towards Case-Optimized Hybrid Homomorphic Encryption Featuring the Elisabeth Stream Cipher*

TFHE: corresponding stream ciphers

Elisabeth-4 [CHMS22] ; $q = 2^4$
 Uses a constant key register on which index-dependent non-linear functions are applied.
 Can be linearized [GBJR23]

Gabriel... [HMS23]
 (Elisabeth-4 follow-ups)

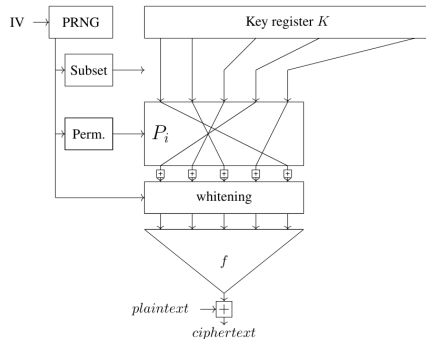


Fig. 1: The group filter permutator design

source: *Towards Case-Optimized Hybrid Homomorphic Encryption Featuring the Elisabeth Stream Cipher*

TFHE: corresponding stream ciphers

Elisabeth-4 [CHMS22] ; $q = 2^4$
 Uses a constant key register on which index-dependent non-linear functions are applied.
 Can be linearized [GBJR23]

Gabriel... [HMS23]
 (Elisabeth-4 follow-ups)

FRAST [CCH⁺24] ; $q = 2^4$ A block cipher in a CTR-mode variant.

See you at the rump session :D

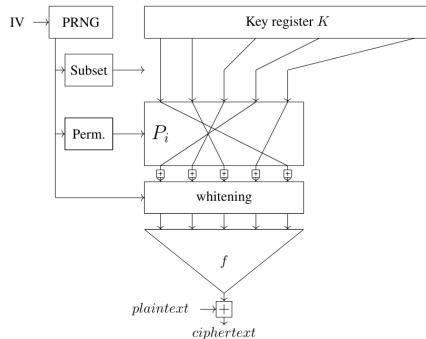


Fig. 1: The group filter permutator design

source: *Towards Case-Optimized Hybrid Homomorphic Encryption Featuring the Elisabeth Stream Cipher*

TFHE: corresponding stream ciphers

Elisabeth-4 [CHMS22] ; $q = 2^4$
 Uses a constant key register on which index-dependent non-linear functions are applied.
 Can be linearized [GBJR23]

Gabriel... [HMS23]
 (Elisabeth-4 follow-ups)

FRAST [CCH⁺24] ; $q = 2^4$ A block cipher in a CTR-mode variant.

See you at the rump session :D

Transistor [BBB⁺25] ; $q = 2^4 + 1$
 SNOW-like round structure
 See you at Anne's invited talk :D

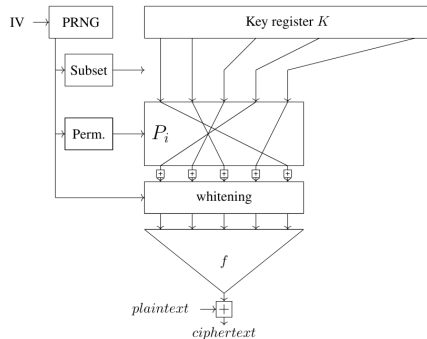


Fig. 1: The group filter permutator design

source: *Towards Case-Optimized Hybrid Homomorphic Encryption Featuring the Elisabeth Stream Cipher*

BGV/FV: corresponding stream ciphers

- ASTA $q = 2$ or large prime
Use very few rounds with a low degree.
Rely on large, randomly generated, nonce-dependent matrices.

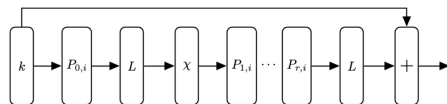


Figure 2: Generation of i -th block of DASTA.

source:

Dasta – Alternative Linear Layer for Rasta

BGV/FV: corresponding stream ciphers

·ASTA $q = 2$ or large prime

Use very few rounds with a low degree.

Rely on large, randomly generated, nonce-dependent matrices.

“Kreyvium” [CCF⁺16] $q = 2$

Basically Trivium!

Binary state updated with NLFSRs.

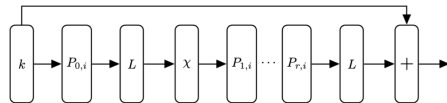


Figure 2: Generation of i -th block of DASTA.

source:

Dasta – Alternative Linear Layer for Rasta

BGV/FV: corresponding stream ciphers

·ASTA $q = 2$ or large prime

Use very few rounds with a low degree.

Rely on large, randomly generated, nonce-dependent matrices.

“Kreyvium” [CCF⁺16] $q = 2$

Basically Trivium!

Binary state updated with NLFSRs.

HERA [CHK⁺21] q large prime

A block cipher in a kind of CTR-mode variant.

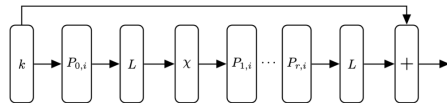


Figure 2: Generation of i -th block of DASTA.

source:

Dasta – Alternative Linear Layer for Rasta

Plan of this Section

[0]

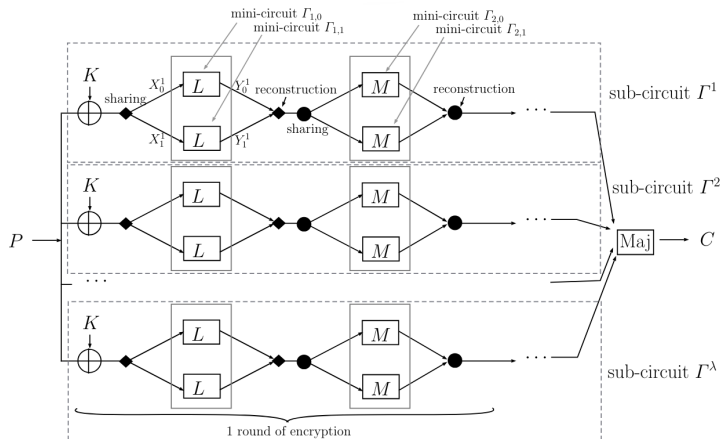
4 Examples of Primitives

■ FHE

■ MPC

■ ZK

Trojan Resilience



source: *MOE: Multiplication Operated Encryption with Trojan Resilience*
<https://tosc.iacr.org/index.php/ToSC/article/view/8834>

MPC-Friendly Encryption

LowMC [ARS⁺15] $q = 2$

SPN with partial layer of quadratic S-boxes.
Rely on large, randomly generated matrices.
Only one encryption/key; broken anyway

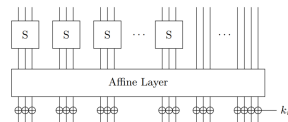


Fig. 1. Depiction of one round of encryption with LowMC.

source:

Ciphers for MPC and FHE

MPC-Friendly Encryption

LowMC [ARS⁺15] $q = 2$

SPN with partial layer of quadratic S-boxes.
Rely on large, randomly generated matrices.
Only one encryption/key; broken anyway

Ciminion [DGGK21] no specific constraints on q

3-branch Feistel network with a single
multiplication/round.

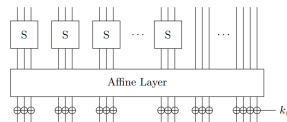


Fig. 1. Depiction of one round of encryption with LowMC.

source:

Ciphers for MPC and FHE

MPC-Friendly Encryption

LowMC [ARS⁺15] $q = 2$

SPN with partial layer of quadratic S-boxes.
Rely on large, randomly generated matrices.
Only one encryption/key; broken anyway

Ciminion [DGGK21] no specific constraints on q

3-branch Feistel network with a single
multiplication/round.

small-pSquare [GMM⁺24a] $p = q = 127$

Generalized Feistel network with low degree round
function.
Optimized specifically for hardware masking.

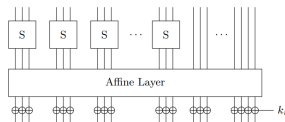


Fig. 1. Depiction of one round of encryption with LowMC.

source:

Ciphers for MPC and FHE

MPC-Friendly Encryption

LowMC [ARS⁺15] $q = 2$

SPN with partial layer of quadratic S-boxes.
Rely on large, randomly generated matrices.
Only one encryption/key; broken anyway

Ciminion [DGGK21] no specific constraints on q

3-branch Feistel network with a single multiplication/round.

small-pSquare [GMM⁺24a] $p = q = 127$

Generalized Feistel network with low degree round function.
Optimized specifically for hardware masking.

MOE [BFL⁺21] $q = 2^{128}$, $m = 2^{128}$

Dedicated structure with linear operations in $\mathbb{F}_q$ and $\mathbb{Z}/q\mathbb{Z}$. Intended for hardware trojan resilience.

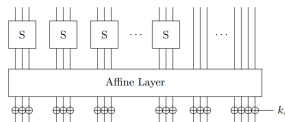


Fig. 1. Depiction of one round of encryption with LowMC.

source:

Ciphers for MPC and FHE

Plan of this Section

[0]

4 Examples of Primitives

- FHE
- MPC
- ZK

ZK-Friendly Hash Functions

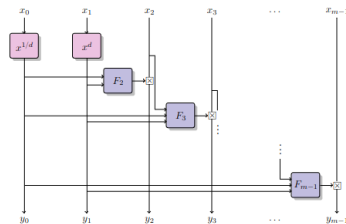
Poseidon [GLR⁺20] no specific constraints q
SPN with partial layer of low degree
monomial S-boxes.
Full rounds – partial round – full rounds.

ZK-Friendly Hash Functions

- Poseidon** [GLR⁺20] no specific constraints q
SPN with partial layer of low degree monomial S-boxes.
Full rounds – partial round – full rounds.
- Rescue** [AAB⁺20] q large prime
SPN with low degree monomials and their inverses.
Most “AES-like”, also most secure at this stage.

ZK-Friendly Hash Functions

- Poseidon** [GLR⁺20] no specific constraints q
 SPN with partial layer of low degree monomial S-boxes.
 Full rounds – partial round – full rounds.
- Rescue** [AAB⁺20] q large prime
 SPN with low degree monomials and their inverses.
 Most “AES-like”, also most secure at this stage.
- Griffin** [GHR⁺23] q large prime
 Feistel variant with multiplications instead.
 Particularly vulnerable to algebraic attacks!

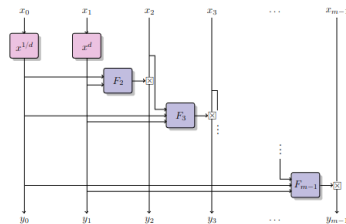


source: *Cryptanalysis and design of symmetric primitives defined over large finite fields*, PhD thesis of C.

Bouvier

ZK-Friendly Hash Functions

- Poseidon** [GLR⁺20] no specific constraints q
 SPN with partial layer of low degree monomial S-boxes.
 Full rounds – partial round – full rounds.
- Rescue** [AAB⁺20] q large prime
 SPN with low degree monomials and their inverses.
 Most “AES-like”, also most secure at this stage.
- Griffin** [GHR⁺23] q large prime
 Feistel variant with multiplications instead.
 Particularly vulnerable to algebraic attacks!
- Anemoi** [BBC⁺23] $q = 2^n$ or large prime
 Uses the “Flystel”, a high degree S-box
 CCZ-equivalent to a function of low degree.



source: *Cryptanalysis and design of symmetric primitives defined over large finite fields*, PhD thesis of C. Bouvier